

EcoHospital Smart

Ciclo CI/CD com Java Spring Boot

Kalicon Amorim da Cruz Souza — RM 563172

FIAP | Atividade acadêmica | 28/09/2026

Código, testes, containerização e configuração de staging/produção.

CI/CD completo verificado: 25 testes JUnit, imagem GHCR, staging e produção Docker no PC com aprovação obrigatória e mesmo digest. Evidências reais anexadas.

Documento gerado a partir de docs/documentacao-tecnica.md. Outros integrantes: preencher se houver.

1. Identificação e objetivo

Integrante: Kalicon Amorim da Cruz Souza — RM 563172. Instituição identificada no projeto original: FIAP.
Outros integrantes: não informados; preencher se houver. Data da preparação: 28/09/2026.

Objetivo: demonstrar build automático, execução de testes existentes, containerização e promoção da mesma versão em staging e produção. CI, testes, build Docker, publicação GHCR e os dois deploys foram executados e verificados. O usuário escolheu este PC para os dois ambientes, sem servidor pago.

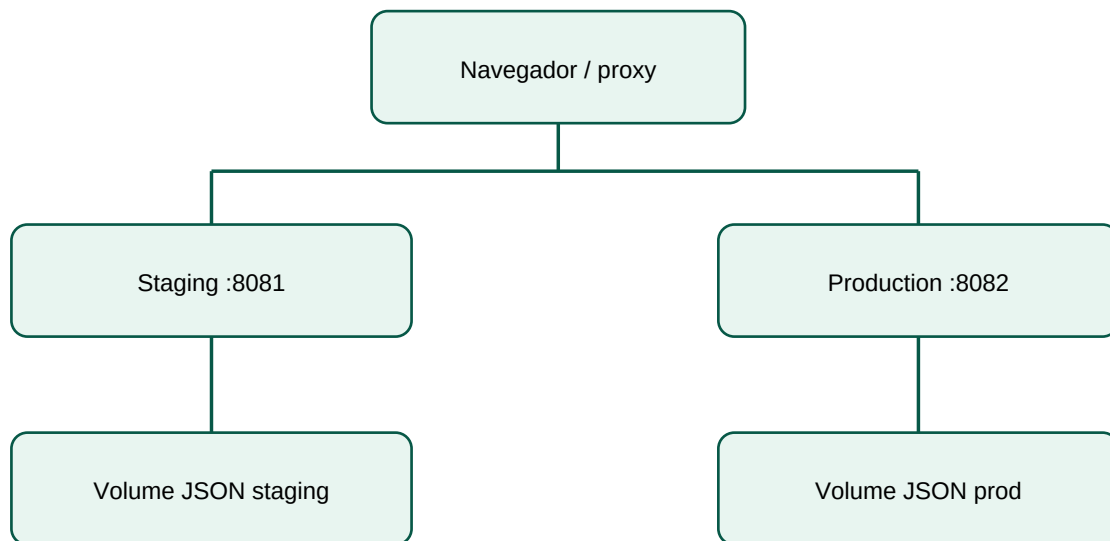
2. Diagnóstico e decisão técnica

O material recebido incluía servidor Node.js nativo, frontend estático, dataset JSON com cinco coleções e scripts acadêmicos MongoDB. Não havia Java/C#, package.json, build, Docker ou CI. A aplicação web simulava consultas MongoDB em arrays e perdia alterações ao reiniciar.

Após autorização para adotar a solução adequada à atividade, foi implementado backend Java 17 com Spring Boot 3.5.16. O frontend e as operações ESG foram mantidos: hospitais, fontes polimórficas, licenças, auditoria, KPIs, consultas, atualizações, simulação IoT e reset. O servidor Node original foi preservado como referência e o runner original continua sendo executado no CI.

Persistência: JSON com gravação atômica em volume por ambiente. Essa escolha mantém o modelo existente com mudanças limitadas e sem serviço de banco artificial. Não se afirma integração MongoDB; seus scripts e prints históricos pertencem à atividade anterior.

3. Arquitetura Docker



Projetos, redes e volumes separados; mesma imagem por digest

Arquitetura prevista: navegador -> proxy/rede do laboratório -> container Spring Boot em cada ambiente -> volume JSON correspondente. O Compose cria uma rede bridge e um volume nomeado por projeto. Staging usa projeto ecohospital-staging e porta 8081; produção usa ecohospital-production e porta 8082. Ambos expõem internamente 8080.

Staging e produção recebem a mesma imagem GHCR por digest SHA256, mas têm configuração e dados separados. Podem estar em dois servidores ou no mesmo host para demonstração, sempre com projetos, portas e volumes distintos. Por padrão, portas no host são limitadas a 127.0.0.1.

O volume contém state.json. O dataset original fica dentro do JAR e é usado somente na criação inicial ou no reset explícito. Reinícios e substituição do container mantêm os dados. Apenas uma instância pode escrever em cada volume. Não há promessa de escalabilidade distribuída, transações MongoDB ou alta disponibilidade.

4. Imagem criada e execução

Dockerfile de dois estágios. Build: maven:3.9-eclipse-temurin-17 executa mvn verify, incluindo JUnit, e gera esg-app.jar. Runtime: eclipse-temurin:17-jre-jammy com JRE, JAR e curl; usuário UID/GID 10001, porta 8080 e healthcheck em /health. Maven e Node não entram no runtime.

O .dockerignore permite apenas pom, Java/testes, frontend e dataset necessário. Compose configura root filesystem somente leitura, tmpfs /tmp, volume de dados gravável, memória de 512 MB, no-new-privileges e capabilities removidas.

Comandos de reprodução: `docker build --build-arg APP_VERSION=academic-local -t ecohospital:local .;`
`docker compose up -d --build --wait --wait-timeout 180.` O pipeline real construiu e testou a imagem; o deploy local usou pull por digest, ``docker compose up --wait`` e health real em 8081/8082.

5. Etapas e lógica do pipeline

Gatilhos: pull request, push na main ou execução manual. verify executa checagem JavaScript, runner histórico com asserções e Maven verify. O JAR e os XML JUnit são guardados como artefatos.

image depende de verify aprovado. Constrói a imagem, executa container, confere usuário, health, versão e persistência após reinício. Na main, publica a imagem testada no GHCR usando GITHUB_TOKEN; o digest real é passado aos dois deploys. Pull requests não publicam nem fazem deploy.

staging depende da imagem e de DEPLOY_ENABLED=true. production depende de imagem e staging e exige PRODUCTION_DEPLOY_ENABLED=true. O Environment production deve ter aprovação configurada no GitHub. Falha em teste, container ou health de staging bloqueia produção pelas dependências needs e pelo código de saída das etapas.

Modo executado: staging-pc e production-pc usam deploy-pc.yml, Docker Desktop Linux e runners Windows efêmeros com label ecohospital-lab. Variables LOCAL_DEPLOY_ENABLED e LOCAL_PRODUCTION_DEPLOY_ENABLED foram habilitadas após o engine ficar pronto. O runner local não executa PRs: exige main e actor Kalicon. Production exigiu e recebeu revisão obrigatória; ambos os Environments estão restritos à main. O pacote GHCR público foi baixado anonimamente por digest, sem PAT permanente. Os runners se removeram após um job cada.

Cada deploy usa secrets do seu Environment: SSH_HOST, SSH_USER, SSH_PORT, SSH_PRIVATE_KEY, SSH_KNOWN_HOSTS, GHCR_USER e GHCR_TOKEN. Variables: APP_PORT, BIND_ADDRESS e APP_URL. A verificação SSH do host é estrita; senha/token não são argumentos de linha de comando.

O servidor faz pull, executa Compose com --wait e confere ambiente e SHA no health. O runner consulta a URL externa e confere os mesmos campos. Evidência JSON é arquivada somente a partir da resposta real. Produção usa o digest de staging sem recompilar. Deploys do mesmo ambiente não ocorrem simultaneamente.

6. Evidências reais disponíveis

Docker Linux 29.8.1 e Compose 5.5.1 responderam neste PC. No run 36438015592, verify, image, staging-pc e production-pc passaram. Kalicon aprovou o Environment production antes do segundo deploy. Ambos usam `ghcr.io/kalicon/ecohospital-esg-cicd@sha256:6a170c3d0f26560afa548a895a67f0f8b8e32481d4a7dc016cc57a7b32b94086` e versão `461ffa3eea37b904018856681c4fbea1cdc2c8b8`. Staging usa porta 8081/volume `ecohospital-staging\_esg-data`; produção usa 8082/volume `ecohospital-production\_esg-data`. Ambos têm health UP e UID 10001. Capturas reais e JSON em docs/evidence/pc.

Teste cruzado real: staging tinha 10 leituras IoT, recebeu uma e ficou com 11; produção permaneceu com 10. Após reiniciar apenas o container staging, manteve 11. JSON `docker-isolation-persistence.json` registra IDs, digest, redes, volumes e contagens. O reinício não apagou nem restaurou volumes.

Duas tentativas anteriores ficaram vermelhas, com produção skipped: primeiro GHCR recusou login do `GITHUB\_TOKEN` para o pacote público; depois DOCKER_CONFIG temporário impediu descoberta do plugin Compose no Windows. A correção foi pull público anônimo sem sobrepor DOCKER_CONFIG. O run de staging aprovado verifica a correção. Essas falhas foram mantidas como evidência do bloqueio de promoção; não foram ocultadas.

Build Maven: aprovado com JDK 17.0.16. JUnit: 25 casos, zero falhas, zero erros e zero ignorados. Runner original: asserções de consultas/CRUD aprovadas para 50 documentos. Logs e XML em docs/evidence/local.

HTTP local: dois processos Java com a mesma versão SHA256 do JAR responderam 200 na página e health. Staging recebeu uma leitura IoT e ficou com 11 leituras; produção permaneceu com 10. Após reinício, staging manteve 11. Esses resultados são reais, porém locais e sem containers.

Actionlint 1.7.12 aprovou os workflows; Git Bash aprovou sintaxe dos scripts SSH. A execução real posterior do Actions aprovou build, smoke test Docker e deploys locais no PC. Não houve deploy SSH porque o usuário escolheu o PC.

Compose standalone 5.5.1 aprovou config dos ambientes sem daemon. As configurações expandidas confirmam portas 8081/8082, redes e volumes com nomes distintos. Esses recursos ainda não foram criados em Docker. A configuração remota foi validada com imagem/digest de exemplo, sem publicação.

CI atualizado aprovado: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/36433415005> . Commit 5b00e1eff60e82993258fdec04263a57fb462e49. Verify e image aprovados; deploys skipped porque ainda desabilitados. Logs, JSON de jobs e captura real em docs/evidence/github.

Imagem GHCR publicada: ghcr.io/kalicon/ecohospital-esg-cicd@sha256:443ba973654cfd1cf9b0992f6f746d0c24eb1ef9b4d4a4562f2580c4eba03a3d . O job image executou container com UID 10001, health UP e ambiente ci; após uma simulação IoT e reinício, preservou 11 leituras no volume. Esses testes ocorreram no runner Ubuntu, não no PC Windows.

Staging real no PC: página e health em <http://localhost:8081>, captura `pc/staging-dashboard.png` e JSON `pc/staging-health.json`. Localhost não é URL pública de hospedagem.

Produção real no PC: aprovação Kalicon registrada em `github/production-approval.json` e captura `pc/github-production-approval.png`; job production-pc aprovado. Página e health em <http://localhost:8082>, captura `pc/production-dashboard.png` e JSON `pc/production-health.json`. Mesmo digest/versão de staging.

Falha proposital comprovada: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/3643347285> . PR 1 / branch demo/test-gate: uma asserção JUnit falhou; image e todos os deploys ficaram skipped. PR encerrado sem merge, preservando a main com testes aprovados. Captura 02-test-gate-failure.png e logs reais arquivados.

7. Desafios e soluções

Linguagem incompatível com o pedido: backend portado para Spring Boot, mantendo contrato de endpoints e dados. Ausência de asserções no runner histórico: adicionadas verificações que geram código não zero em divergência. Perda de dados no reinício: snapshots e substituição atômica de arquivo por ambiente.

Primeiro teste da página falhou porque MockMvc retorna forward sem renderizar o destino. Corrigido para verificar forward e index.html; também executado HTTP real. A execução final dos 25 testes passou.

Ferramentas indisponíveis: Maven resolvido com Wrapper e checksum; JDK 17 existente selecionado. Actions atualizadas após avisos do primeiro run. Docker Desktop oficial instalado com assinatura Authenticode verificada; WSL 2.7.14 habilitado após aprovação UAC e reinicialização manual do usuário. Dois deploys reais foram então executados. Login GHCR recusado e conflito de DOCKER_CONFIG com Compose causaram runs vermelhos; o pacote público passou a ser baixado anonimamente sem modificar configuração do Docker. O CI bloqueou produção nessas falhas.

Interface melhorada: identificação do ambiente e da versão no painel; requisições HTTP com erro não são mais apresentadas como sucesso; contadores não substituem zero por um valor fictício. Funcionalidades ESG e dataset original permanecem preservados.

8. Operação, limitações e entrega

A entrega original não tinha autenticação nas rotas didáticas de mutação/reset; a revisão posterior acrescenta token de operador, conforme seção 10. Não usar como serviço público de produção sem identidade individual, TLS e autorização apropriados. A revisão visual usa fontes do sistema e fotografia empacotada, sem Google Fonts externo.

Rollback manual preserva volume e usa previous-image.txt do ambiente. Falha de deploy exige diagnóstico; não há rollback automático, zero downtime ou backup remoto automatizado. Dados devem ser copiados antes de mudanças de formato.

Conclusão: Windows reiniciado manualmente, engine Linux verificado, runners efêmeros usados, variables locais habilitadas, staging executado, produção revisada e executada, capturas e logs reais registrados. Não é necessário servidor pago ou SSH para o modo PC. Localhost só funciona enquanto este computador e os containers estiverem ligados. Se a rubrica exigir acesso público externo, essa parte ainda depende de hospedagem/proxy e credenciais próprios; não foi alegada.

ZIP inclui código Java/Node, dados, frontend, Docker/Compose, workflows, Wrapper, scripts, exemplos e documentação/evidências disponíveis; exclui segredos, ferramentas e estados runtime. O manifesto SHA256 permite conferir conteúdo.

9. Referências oficiais

Spring Boot 3.5 — requisitos: <https://docs.spring.io/spring-boot/3.5/system-requirements.html>

GitHub — environments e proteção:

<https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>

GitHub — aprovação e disponibilidade por plano: <https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments/review-deployments>

10. Evolução de segurança, dados e análise ESG

Em revisão posterior à entrega original, foram acrescentadas três melhorias opt-in, sem alterar os volumes existentes. Primeiro, todas as operações que mudam estado — simulação, reset e dois presets update — exigem token de operador por cabeçalho; leitura permanece disponível. O token tem no mínimo 32 caracteres, é comparado por hash em tempo constante e é montado como arquivo secreto no Compose. O painel permite ativar o token apenas na memória da aba e confirma reset explicitamente. Cabeçalhos CSP, no-sniff, no-frame e no-store reduzem a superfície do navegador. O token compartilhado não substitui identidade individual, TLS ou firewall em uma hospedagem pública.

Segundo, `APP_STORAGE=postgres` habilita PostgreSQL 17 com documento JSONB transacional, linha única bloqueada com `SELECT FOR UPDATE` durante escritas e volume persistente sem porta de banco exposta ao host. Na primeira inicialização, o aplicativo importa `state.json` se presente e válido; em reinícios, o estado no banco prevalece. JSON continua sendo o padrão e nenhum volume anterior é apagado. A cópia do staging foi importada em um terceiro Compose isolado em `localhost:8083`: 11 leituras na origem, HTTP 401 sem token, 12 após escrita autenticada e reinício. Essa comprovação é local; não se deve apresentá-la como migração de staging/produção até um deploy validado.

Terceiro, `GET /api/insights` calcula taxa de conformidade, alertas, violações, licenças críticas e ranking de fontes por média de CO2 em relação ao limite cadastrado. A fórmula é mostrada na interface. Os números são descritivos dos registros presentes, não uma certificação ESG nem dados medidos fora da simulação. Em teste local desta revisão, os 30 casos JUnit passaram; o build Docker anterior à inclusão dos três testes extras aprovou 27. O novo smoke test PostgreSQL do CI ainda precisa de execução no GitHub para ser marcado como comprovado.

11. Redesenho visual

A referência principal Monsoon orientou a paisagem clara, navegação em cápsula translúcida e abertura editorial. A implementação mantém o frontend estático integrado ao Spring Boot, com atalhos para abas reais do painel. Ícones SVG substituem emojis, erros ficam junto às ações, a confirmação de reset usa dialog nativo e o movimento reduzido é respeitado. Datas e especificações usam os campos reais do dataset, sem valores fictícios quando ausentes.

A fotografia é de A.T.M. Arafath Ali / Unsplash

(https://unsplash.com/photos/misty-hills-with-trees-at-sunrise-mGp2_4MeGIw), sob licença Unsplash

(<https://unsplash.com/license>). O vídeo Monsoon foi excluído da entrega devido às restrições de

reutilização de filmagens de demonstração nos termos Scrolltide. A revisão deve ser validada em

localhost:8083 antes de promoção para staging e produção; não altera retroativamente as evidências dos deploys anteriores.

12. Propósito operacional: atenção, análise e acompanhamento

O painel evolui de exposição de indicadores para uma rotina de gestão ambiental: central de atenção por unidade, consulta aos registros de origem e plano de ação. A central usa a última leitura com timestamp válido por fonte (não a ordem do array), sinaliza registros fora de 24 horas ou sem data e confere licenças pelo calendário UTC, com atenção em até 30 dias. Essa janela é didática e não substitui exigências regulatórias. Históricos continuam visíveis e uma tarefa concluída não apaga o alerta original.

POST /api/actions cadastra título, unidade existente, equipe e prazo ISO. PATCH /api/actions/{id} atualiza andamento e exige registro de análise para concluir; ambas usam o token de operador. GET /api/actions e /api/operations são de leitura. O histórico da tarefa mantém status, instante e registro informado; não comprova autoria individual porque o token é compartilhado, nem valida automaticamente a evidência. Ações ficam na lista opcional planos_acao do estado persistente JSON/JSONB, sem exigir alteração das cinco coleções originais. Reset elimina também as ações do ambiente.

Metas não são resultados. Média histórica de kg/h não é um inventário anual. Árvores sugeridas por leituras não são plantios realizados, remoção medida ou créditos certificados. Dados continuam demonstrativos; não há pacientes, sensores físicos, upload de comprovantes ou notificações. A promoção deste incremento a staging/produção requer novo pipeline validado.

13. Inventário GEE e rastreabilidade de resíduos

O inventário registra consumo mensal, escopo 1/2/3, unidade, fator em kgCO₂e por unidade, fonte, versão/base GWP, limite/metodologia, referência de evidência e equipe declarante. O cálculo usa BigDecimal e preserva o fator original. Escopo 2 mantém localização e mercado em resultados separados; não soma métodos alternativos. Não há fator oficial predefinido ou transformação automática das leituras kg/h em emissão anual. O resumo anual é parcial e declarado; ausência de lançamentos não significa emissão zero.

O registro de resíduos preserva lote, massa, data, grupo A–E, setor, manejo, prestador, destino e referências de transporte/destinação. Uma referência ausente gera pendência; uma referência preenchida não equivale à validação do documento. Grupos, subgrupos e condições reais precisam de avaliação técnica no PGRSS. Não se presume descarte comum após autoclavagem, nem redução percentual fixa. Há exportação JSON por recorte de ano e unidade, incluindo registros anulados.

POST `/api/journal/inventory` e `/waste` exigem token. Anulação por POST `/api/journal/{kind}/{id}/void` exige motivo e equipe, preservando o lançamento original e retirando-o dos totais da interface. Correção exige novo registro; referência ativa duplicada por unidade/método é rejeitada. IDs diferentes para o mesmo consumo ainda podem causar sobreposição; revisão técnica continua necessária. As listas opcionais ficam no mesmo JSON/JSONB transacional. Reset remove também ações e novos registros; backup obrigatório antes do uso. Administrador do banco pode alterar o estado; histórico não é criptograficamente inviolável e token compartilhado não comprova identidade individual.

Fontes: GHG Protocol (<https://ghgprotocol.org/calculation-tools-faq> e <https://ghgprotocol.org/scope-2-guidance>), MCTI (<https://www.gov.br/mcti/pt-br/acompanhe-o-mcti/sirene/dados-e-ferramentas/fatores-de-emissao>), Anvisa RDC 222/2018 comentada (<https://www.gov.br/anvisa/pt-br/centraisdeconteudo/publicacoes/servicosdesau/publicacoes/rdc-222-de-marco-de-2018-comentada.pdf/@@download/file>). O aplicativo não declara conformidade integral com essas referências.

Verificação local: 39 testes JUnit aprovados em Maven e no build Docker Java 17. Detalhes e relatórios desta revisão em `docs/evidence/v3`. Evidências de deploy anteriores são mantidas e não representam promoção automática deste incremento.

Anexo — resultados locais reais

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	20	0	0	0
EsgStoreTest	5	0	0	0

HTTP local — staging

```
{"environment": "staging", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "status": "UP"}
```

HTTP local — production

```
{"status": "UP", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "environment": "production"}
```

Verificado em 2026-09-28T10:55:39.2874294-03:00: HTTP 200 em / e /health dos dois processos; versões SHA256 iguais; dados isolados (11/10); persistência após reinício (11). Demonstração local Java, sem Docker ou deploy remoto.

Os resultados acima são da primeira demonstração Java local. As próximas capturas mostram o GitHub Actions e os containers Docker reais no PC.

Anexo — evolução local verificada

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	22	0	0	0
EsgStoreTest	5	0	0	0
WriteAccessTest	3	0	0	0

PostgreSQL isolado no PC: {"checkedAtUtc": "2026-09-28T15:19:55.6166403Z", "environment": "postgres-demo", "storage": "PostgreSQL JSONB", "health": "UP", "readingsBeforeRestart": 12, "unauthorizedResetHttp": 401, "readingsAfterRestart": 12, "databaseContainer": "ecohospital-postgres-demo-db-1", "appContainer": "ecohospital-postgres-demo-app-1"}

Esta prova é do ambiente postgres-demo, não da migração de staging/produção.

Anexo — revisão operacional e rastreabilidade

Suíte	Testes	Falhas	Erros	Ignorados
EnvironmentalJournalTest	3	0	0	0
EsgApiTest	24	0	0	0
EsgOperationsTest	4	0	0	0
EsgStoreTest	5	0	0	0
WriteAccessTest	3	0	0	0

Resultados locais desta revisão. Não constituem promoção no CI/CD nem comprovação ambiental.

```
{ "at": "2026-09-28T16:11:27.6558190Z", "environment": "postgres-demo", "storage": null, "unauthorized": 401, "duplicate": 400, "syntheticCalculationKgCO2e": 123.456789, "inventoryVoidedAndPersisted": true, "wasteMissingProofDetected": true, "wasteSyntheticVoided": true, "actionPersisted": true, "actionHistoryEntries": 2, "readingsBefore": 12, "readingsAfter": 12, "note": "Somente teste tecnico sintetico em postgres-demo; nenhum resultado ambiental real comprovado." }
```

Captura real — postgres-demo

 **EcoHospital Smart**
Inteligência ambiental hospitalar

PostgreSQL JSONB

Arquivos - postgres-demo

Token de edição

Ativar edição

+ Simular leitura

Validar dados

Restaurar dados

PAINEL

Visão geral

Unidades10

Fontes de emissão10

Telemetria IoT12

Licenças10

Governança10

Plano de ação2

Inventário GEE

Restrear resíduos

FERRAMENTAS

Consultas

ODS relacionadas

ODS 3 ODS 11 ODS 13

ODS 15

Inventário de gases de efeito estufa

Consumo x fator = kgCO₂e. Registros declarados, sem verificação independente. As leituras históricas em kg/h não são convertidas em inventário anual.

Informe fator em kgCO₂e por unidade, com GWP já incorporado, fonte e versão. Não multiplique o GWP novamente. Não há fatores oficiais pré-carregados. Gases anestésicos exigem metodologia e fatores específicos, revisados por profissional habilitado.

Recorte por unidade

Toda a rede

Ano do registro

2026

Exportar registros deste recorte (JSON)

Inventário parcial declarado (2026) - 0 lançamentos ativos. Escopo 1: 0 kgCO₂e - Escopo 2 localização: 0 kgCO₂e - Escopo 2 mercado: 0 kgCO₂e - Escopo 3: 0 kgCO₂e. Métodos do escopo 2 são alternativos, não somados. Ausência de registro não significa emissão zero.

Registrar consumo e fator

Unidade

Hospital Central de Clínicas Sustentáveis

Período do consumo

----- de -----

Escopo

2

Método do escopo 2

Localização

Atividade / categoria

Unidade do consumo

TESTE-TECNICO-GEE-
57b6563fd1384e838078b86617e122f5

Hospital Central de Clínicas Sustentáveis - 2026-09

Escopo 2 - Localização - TESTE SINTETICO - não representa consumo hospitalar: 1.000 kWh x 0,123456789 kgCO₂e/kWh = 123,456789 kgCO₂e

Fator preservado: FATOR SINTETICO exclusivo para testar a multiplicação, sem validade ambiental - teste técnico v1

Limite / método: Dados fictícios de teste, não usar em relatórios reais

Equipe declarante: Equipe de testes (demonstração) - registrado em 26/09/2026, 13:11:17 - ID a1032604-b405-45c8-b527-0630b130e8ce

ANULADO em 2026-09-26T16:11:17.567018765Z por Equipe de testes: Encerramento de teste técnico sintético - excluir dos totais

TESTE-TECNICO-GEE-
5b5bc0692dbf4ff690b901a6964e0e72

Hospital Central de Clínicas Sustentáveis - 2026-09

Escopo 2 - Localização - TESTE SINTETICO - não representa

EcoHospital | CI/CD acadêmico | Preparação: 28/09/2026

19

Captura real — postgres-demo



EcoHospital Smart
Inteligência ambiental hospitalar

PostgreSQL: ISO 9001

Antes: postgres-demo

Token de edição

Ativar edição

+ Simular leitura

Validar dados

Restaurar dados

PAINEL

Visão geral

Unidades10

Fontes de emissão10

Telemetria IoT12

Licenças10

Governança10

Plano de ação2

Inventário GEE

Restrear resíduos

FERRAMENTAS

Consultas

ODS relacionadas

ODS 3 ODS 11 ODS 13

ODS 15

Central de atenção ambiental

Primeiro entenda a prioridade; depois confira o registro e organize uma ação. Os indicadores acima e as seções históricas abaixo abrangem toda a rede.

O que precisa de atenção?

Unidade da central

Hospital Central de Clínicas Sustentáveis

Prioridades e próximos passos

6 pontos de atenção - 2 prioritários - Hospital Central de Clínicas Sustentáveis. Cobertura da rede: 10/10 fontes com leitura datada; 0 fora da janela de atualização. Análise: 28/09/2026, 13:11:33.

Prioridade alta

Caldeira Aquotubular Principal - Ala Norte

Hospital Central de Clínicas Sustentáveis

Última leitura registrada: 162010 kg/h - VÍCIACAO_BLOQUEANTE - registro histórico

Próximo passo sugerido: Revisar o registro e a condição do equipamento com a equipe técnica; documentar a análise.

Consultar registros Planejar ação

Prioridade alta

CETESB-2024-SP-098234

Hospital Central de Clínicas Sustentáveis

Prazo cadastrado: 2026-04-15 - vencido pelo calendário

Próximo passo sugerido: Conferir a validade documental com a equipe ambiental e acompanhar a renovação.

Consultar registros Planejar ação

Analisar dados

Caldeira Aquotubular Principal - Ala Norte

EcoHospital | CI/CD acadêmico | Preparação: 28/09/2026

20

Anexo — captura real do GitHub

Run inicial: verify e image aprovados; naquele momento os deploys ainda estavam desabilitados.

The screenshot shows the GitHub Actions interface for the repository 'Kalicon / ecohospital-esg-cicd'. The workflow 'EcoHospital CI/CD' is displayed, with the title 'Atualiza Actions e prepara deploy Docker em dois ambientes no PC #2'. The status is 'Success', with a total duration of '2m 0s' and '3' artifacts. The workflow diagram shows a sequence of steps: 'verify' (green), 'image' (green), 'staging-pc / deploy' (grey), and 'production-pc / deploy' (grey). The 'verify' and 'image' steps are marked as successful, while the deployment steps are greyed out, indicating they are disabled.

Summary

Triggered via push 5 minutes ago

Kalicon pushed [5b00e1e](#) [main](#)

Status	Total duration	Artifacts
Success	2m 0s	3

ci-cd.yml

on: push

```
graph LR; verify[verify] --> image[image]; image --> staging_deploy[staging-pc / deploy]; image --> production_deploy[production-pc / deploy]; staging_deploy --> staging_deploy; production_deploy --> production_deploy;
```

Origem: docs/evidence/github/03-updated-ci-success.png. Captura da página real, sem montagem de status.

Anexo — captura real do GitHub

PR descartável: verify falhou; image e deploys bloqueados. PR fechado sem merge.

The screenshot shows the GitHub interface for the repository 'Kalicon / ecohospital-esg-cicd'. The 'Actions' tab is selected, displaying a workflow named 'EcoHospital CI/CD'. The workflow is in a 'Failure' state, indicated by a red 'x' icon and the title 'Demonstração acadêmica: testes falhos bloqueiam imagem e deploy #3'. The summary shows it was triggered via a pull request 2 minutes ago, opened by 'Kalicon' with the PR number '#1' for the branch 'demo/test-gate'.

Status	Total duration	Artifacts
Failure	41s	1

The workflow diagram, titled 'ci-cd.yml on: pull_request', shows a sequence of steps: 'verify' (failed), 'image', 'staging-pr / deploy', and 'production-pr / deploy'. The 'verify' step is marked with a red 'x' and a duration of 11s. The 'image' step is marked with a red 'x' and a duration of 30s. The 'staging-pr / deploy' and 'production-pr / deploy' steps are marked with red 'x' icons. The workflow is shown as a sequence of steps connected by arrows, with the 'verify' step failing and blocking the subsequent steps.

Origem: docs/evidence/github/02-test-gate-failure.png. Captura da página real, sem montagem de status.

Anexo — ambiente Docker no PC

Run final aprovado: verify, image, staging-pc e production-pc.

The screenshot shows a GitHub Actions workflow run for the repository 'Kalicon / ecohospital-esg-cicd'. The workflow is named 'Preserva descoberta do plugin Compose no runner' and is running on a Windows environment. The run is successful, with a status of 'Success' and a total duration of 10m 8s. The workflow is triggered via a push to the main branch.

Workflow Summary:

- Status: Success
- Total duration: 10m 8s
- Artifacts: 5

Workflow Steps:

```
graph LR; verify[verify] --> image[image]; image --> staging[staging-pc / deploy]; image --> production[production-pc / deploy]; staging --> staging_deploy[staging / deploy]; production --> production_deploy[production / deploy];
```

The workflow steps are: verify, image, staging-pc / deploy, production-pc / deploy, staging / deploy, and production / deploy. The first four steps are successful, while the last two are pending.

Origem: docs/evidence/pc/github-two-deploys-approved.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

Histórico real: Kalicon aprovou production antes do deploy.

The screenshot shows a deployment interface with a top section containing two buttons: "staging / deploy" and "production / deploy". Below this is a section titled "Deployment protection rules" with the subtitle "Reviewers, timers, and other rules protecting deployments in this run". A table follows with two columns: "Event" and "Environments". The "Event" column contains a user profile for "Kalicon" with a green checkmark and the text "approved 1 minute ago". The "Environments" column contains the word "production". Below the table is a section titled "Artifacts" with the subtitle "Produced during runtime". A table follows with three columns: "Name", "Size", and "Digest". The "Name" column contains "Kalicon~ecohospital-esg-cicd~FG1XSG.dockerbuild". The "Size" column contains "41.2 KB". The "Digest" column contains "sha256:a0691750ca123..." and a copy icon. At the bottom, there is a section titled "build-and-tests-" with a progress bar.

Event	Environments
 Kalicon approved 1 minute ago	production

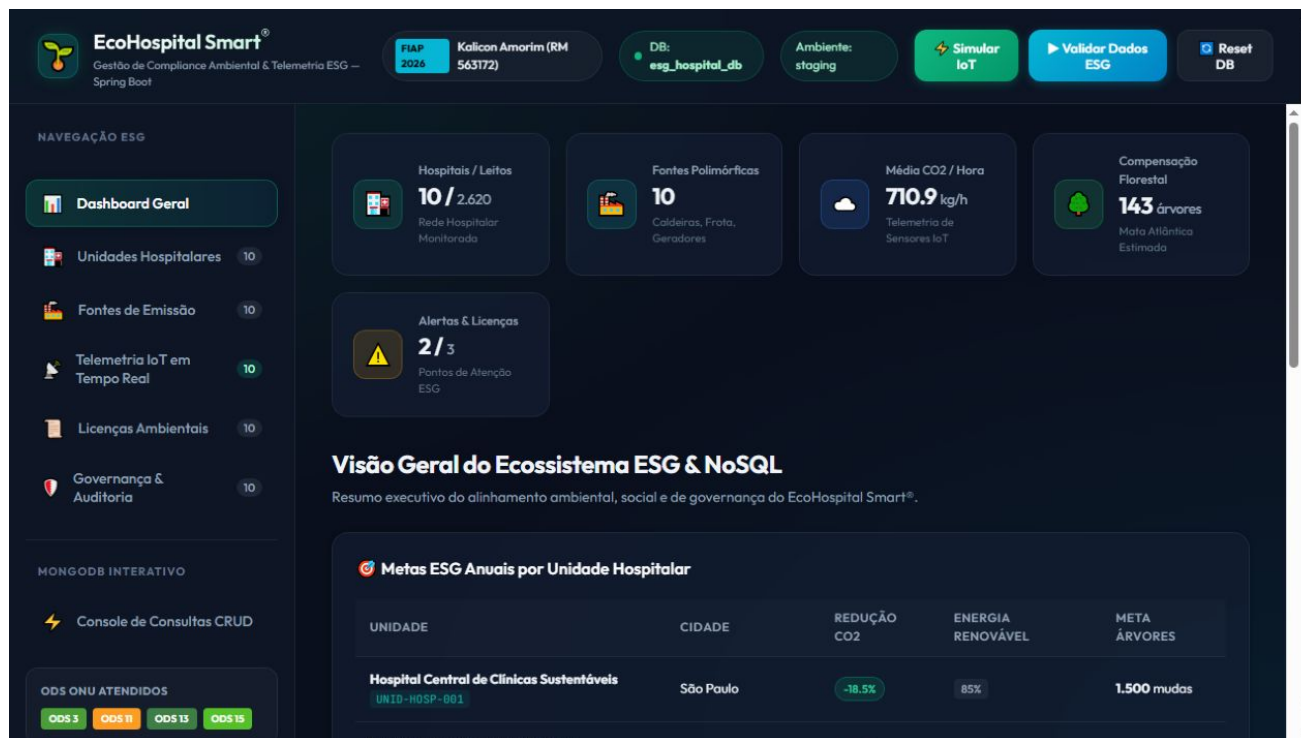
Name	Size	Digest
 Kalicon~ecohospital-esg-cicd~FG1XSG.dockerbuild	41.2 KB	sha256:a0691750ca123... 

build-and-tests-

Origem: docs/evidence/pc/github-production-approval.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

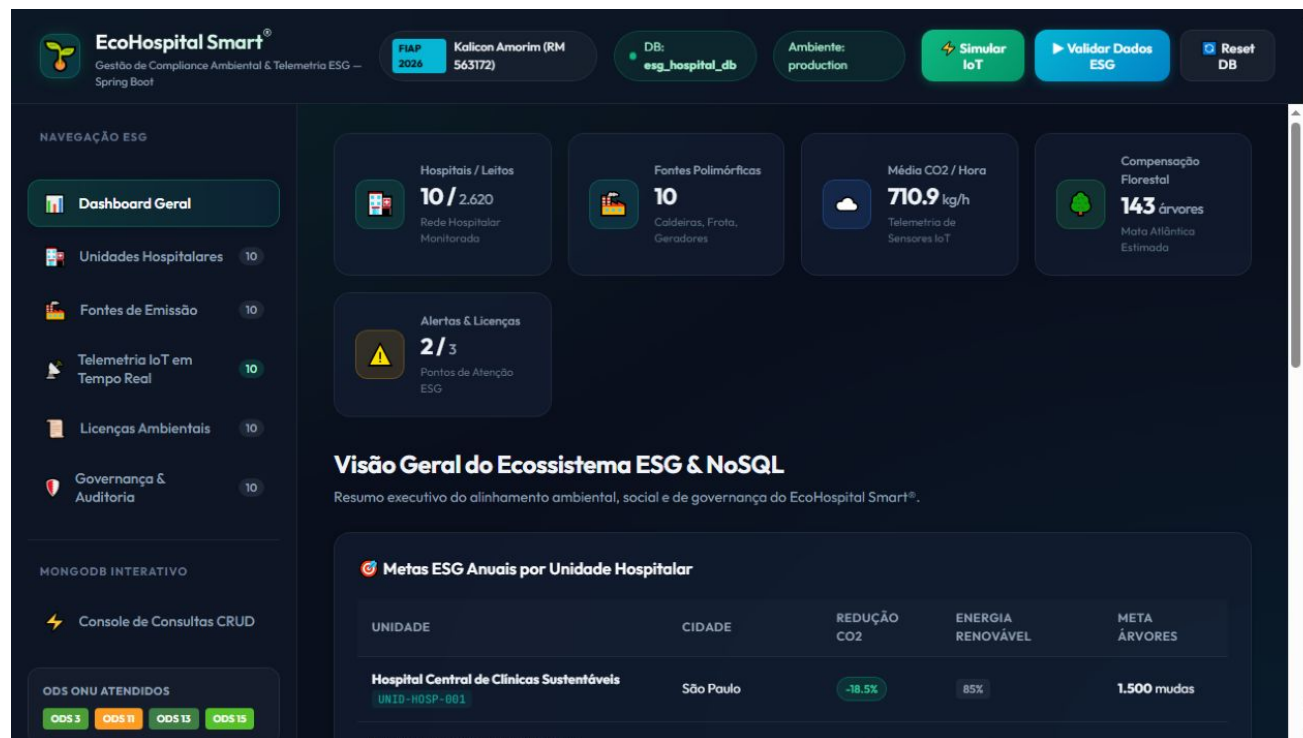
Dashboard real de staging no container Docker deste PC, porta 8081.



Origem: docs/evidence/pc/staging-dashboard.png. Captura real; URLs localhost não são servidores públicos.

Anexo — ambiente Docker no PC

Dashboard real de produção no container Docker deste PC, porta 8082.



Origem: docs/evidence/pc/production-dashboard.png. Captura real; URLs localhost não são servidores públicos.